

Conventional Commits

This format adds a "label" or prefix to the message to indicate the type of change. Its basic structure is:

```
type (optional scope): brief description in the imperative
```

The most common types are:

- **feat**: When you add new functionality to the project (e.g., feat: add remote sensing data extraction module).
- **fix**: When you fix an error or "bug" (e.g., fix: resolve coordinate projection mismatch in Python script).
- **docs**: When you modify or add documentation, such as the README.md file (e.g., docs: update methodology description in README).
- **refactor**: When you reorganize or clean up code without changing its functionality (e.g., refactor: simplify raster processing functions).
- **test**: When you add or fix tests (e.g., test: add unit tests for land suitability algorithm).
- **chore**: General maintenance tasks, dependencies, or configuration files (e.g., chore: update requirements.txt).

The goal of these best practices is that, by quickly glancing at the commit history, you'll know exactly which document was added or modified without having to open each commit

Ideally, specify the scope in parentheses or detail the document's content in the short description.

How would this look in practice?

```
* docs(readme): add project installation instructions
* docs(methodology): include remote sensing data workflow
* docs(colombia-case): add study area description for Cauca region
* feat(pipeline): add spatial filter for raster data
* feat(export): enable GeoTIFF export function
```

1. The parentheses clarify the context: docs(readme) or docs(methodology) immediately tells you which part of the documentation was updated.
2. It's unique and descriptive: If you search the history to find when you wrote the methodology, you'll find it in seconds thanks to the specific tag

(Spanish)

Este formato le da una "etiqueta" o prefijo al mensaje para que se sepa de qué tipo es el cambio. Su estructura básica es:

```
type (optional scope): brief description in the imperative
```

Los tipos más comunes son:

- **feat**: Cuando agregas una nueva funcionalidad al proyecto (ej. feat: add remote sensing data extraction module).
- **fix**: Cuando corriges un error o "bug" (ej. fix: resolve coordinate projection mismatch in python script).
- **docs**: Cuando modificas o agregas documentación, como el archivo README.md (ej. docs: update methodology description in README).
- **refactor**: Cuando reorganizas o limpias código sin cambiar lo que hace (ej. refactor: simplify raster processing functions).
- **test**: Cuando añades o corriges pruebas (ej. test: add unit tests for land suitability algorithm).
- **chore**: Tareas de mantenimiento general, dependencias o archivos de configuración (ej. chore: update requirements.txt).

El objetivo de las buenas prácticas es que, al mirar el historial rápidamente, sepas exactamente qué documento se agregó o modificó sin tener que abrir cada commit.

Lo ideal es especificar el alcance (scope) entre paréntesis o detallar el contenido del documento en la descripción corta.

1 ¿Cómo se vería en la práctica?

```
* docs(readme): add project installation instructions
* docs(methodology): include remote sensing data workflow
* docs(colombia-case): add study area description for Cauca region
* feat(pipeline): add spatial filter for raster data
* feat(export): enable GeoTIFF export function
```

1. El paréntesis aclara el contexto: docs(readme) o docs(methodology) te dice de inmediato qué parte de la documentación se tocó.
2. Es único y descriptivo: Si buscas en el historial cuándo escribiste la metodología, lo vas a encontrar en segundos gracias a la etiqueta específica.